02 / GUÍA DE REFERENCIA PYTHON

Herramientas NumPy para rotaciones 3D

EX30 · Funciones, entradas, salidas y ejemplos breves

Objetivo. Esta guía describe las herramientas de NumPy necesarias para construir y verificar matrices de rotación 3D. Para cada herramienta se indican entradas, salidas, ejemplos breves y errores típicos. No es la solución del EX30.

1. `np.deg2rad()`: conversión de ángulos

Recibe: un número o array en grados. **Devuelve:** el ángulo correspondiente en radianes, como escalar o array. Seno y coseno de NumPy reciben radianes.

```
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
# angle_rad ~ 1.5707963268
```

Error habitual: usar `np.cos(90)` pensando que representa $\cos 90°$. En realidad son 90 radianes.

2. `np.sin()`, `np.cos()`: entradas trigonométricas

Reciben: ángulos en radianes (escalares o arrays). **Devuelven:** valores numéricos con la misma forma de entrada.

```
c = np.cos(np.deg2rad(90))  # approx. 0
s = np.sin(np.deg2rad(90))  # approx. 1
```

Nota numérica: $\cos 90°$ puede aparecer como $6{,}12 \times 10^{-17}$, una aproximación a cero debida a la precisión finita del ordenador.

3. `np.array()`: construir una matriz 3 por 3

Recibe: una secuencia de tres filas, cada una con tres números. **Devuelve:** un `ndarray` de forma `(3, 3)`. Aquí construimos una matriz independiente del ejercicio para un ángulo α:

```
import numpy as np
alpha = np.deg2rad(45.0)
c = np.cos(alpha)
s = np.sin(alpha)
Rz = np.array([
    [c, -s, 0.0],
    [s,  c, 0.0],
    [0.0, 0.0, 1.0]
])
print(Rz.shape)  # (3, 3)
```

Regla de programación: al cambiar el ángulo, recalcular c, s y **reconstruir** la matriz; cambiar solo la variable no modifica una matriz que ya existe.

4. `@`: multiplicación matriz-vector y matriz-matriz

Recibe: arrays con dimensiones compatibles. **Devuelve:** su producto matricial. Una matriz `(3, 3)` por un vector `(3,)` produce un vector `(3,)`.

```
I = np.eye(3)
v = np.array([2.0, -1.0, 4.0])
w = I @ v            # [2., -1., 4.]
print(w.shape)       # (3,)
```

Recuerda: `R2 @ R1 @ v` aplica primero `R1` y después `R2`. `R1 * R2` realiza multiplicación *elemento a elemento*; no compone rotaciones.

5. `.T` y `np.eye(3)`: transpuesta e identidad

`.T`: recibe implícitamente un array y devuelve una *vista* con sus ejes transpuestos. Para una matriz 3 por 3 intercambia filas y columnas.

`np.eye(3)`: recibe la dimensión 3 y devuelve la identidad $\mathbf{I}_3$, con unos en la diagonal y ceros fuera.

```
I = np.eye(3)
check_matrix = Rz.T @ Rz
print(np.allclose(check_matrix, I))  # True
```

Si `Rz` es una rotación propia, se cumple $R_z^T R_z \simeq I_3$ y `Rz.T` invierte el giro.

6. `np.allclose()` frente a `np.isclose()`

`np.allclose(a,b)`: compara dos arrays numéricos con tolerancia y devuelve **un único booleano**. **`np.isclose(a,b)`:** devuelve un array de comparaciones elemento a elemento.

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([1.0, 2.0 + 1e-12, 3.0])
print(np.allclose(a, b))  # True
print(np.isclose(a, b))   # [ True  True  True]
```

Por qué es útil: evita comparar de forma exacta resultados trigonométricos calculados en coma flotante.

7. `np.linalg.norm()`, `np.linalg.det()` y `np.cross()`

Norma: `np.linalg.norm(v)` recibe un vector y devuelve su norma euclídea. Por ejemplo, `np.linalg.norm([3,4,0])` devuelve 5.

Determinante: `np.linalg.det(R)` recibe una matriz cuadrada y devuelve un escalar. Para una rotación 3D propia se espera aproximadamente $+1$.

Producto vectorial: `np.cross(r, F)` recibe dos vectores tridimensionales y devuelve un vector perpendicular, por ejemplo el momento $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}$ (EX27).

```
r = np.array([0.2, 0.0, 0.0])  # m
F = np.array([0.0, 0.0, 10.0]) # N
M = np.cross(r, F)             # [ 0., -2., 0.] N*m
```

Importante: comprobar que $\det R \simeq 1$ no sustituye a la condición $R^T R \simeq I$.

8. Lista de comprobación antes de ejecutar

1. Los tres ángulos se calculan en radianes; se construyen nuevas matrices al cambiar los ángulos.
2. Cada matriz tiene forma `(3, 3)` y cada vector tiene forma `(3,)`.
3. Se respetan los signos de R_x, R_y y R_z y el orden de multiplicación con `@`.
4. Los resultados y sus normas se comparan con los valores manuales, usando tolerancias numéricas.
5. Las unidades se imprimen para fuerzas (N), brazos (m) y momentos (N m).